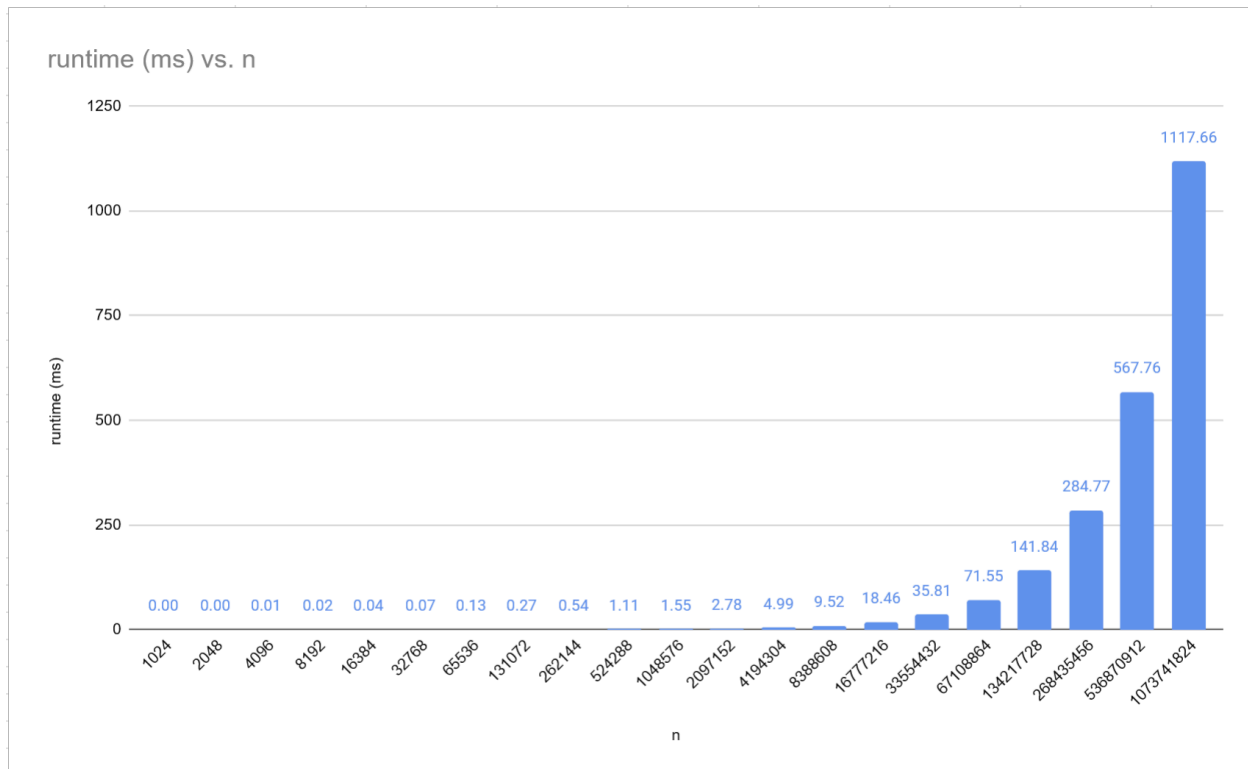


1) Here is the plot acquired during the runs.



2) Done. See GitHub repo.

3) I got times of **773.67ms**, **386.57ms**, **1100.47ms** and **771.44ms** respectively for **n=1000** on **Euler**. mmul1, 2 and 3 all have differing performance due to how we index into their arrays. mmul1 operates in the middle of the 3, as we iterate over rows in A, columns in B and then iterate through the specified column, summing the value and writing to C. mmul2 has the best performance. Looking at how we index into the array, we see we actually are doing a stride of only a single value across C and B instead of whole rows (like in mmul1) and A stays fixed for the entire inner loop. This means we are always doing sequential accesses within the innermost loop. In mmul3, we perform the worst since in the innermost loop we take huge strides in arrays A and C each loop and are jumping entire rows/columns. mmul1 and mmul4 both perform the same basic operation and operate essentially the same. Since both are allocated as contiguous memory blocks*, the time it takes to index through them is the same barring minor execution differences.

*During Claude Opus 5.5 check, my understanding was corrected and I was informed that vectors are assigned as contiguous memory blocks like in a 1D array.

